

Obsah

1	Mysqli.....	3
2	PDO (PHP Data Objects).....	3
2.1	Jaké přináší výhody a jak ho používat.....	3
2.1	Narušení SQL dotazu (SQL injection).....	4
2.1.1	Řešení.....	4
2.2	PDO Připojení k MySQL.....	6
2.3	Obyčejný dotaz.....	6
2.4	Předpřipravený dotaz (prepared statement).....	7
2.4.1	SELECT.....	7
2.4.2	INSERT.....	7
2.4.3	UPDATE.....	9
2.4.4	DELETE.....	9
2.5	Procházení dat.....	9
2.5.1	Více řádků.....	9
2.5.2	Jeden řádek.....	9
2.5.3	Jedena položka.....	10
2.6	Vícenásobný INSERT.....	10
2.7	Příklad PDO.....	12
2.8	Přidání dat pomocí PDO.....	13
2.9	Mazání dat pomocí PDO.....	13
2.10	Oprava dat pomocí PDO.....	14
3	REST API.....	15
3.1	Metody pro přístup ke zdrojům.....	15
3.1.1	GET (Retrieve).....	15
3.1.2	POST (Create).....	15
3.1.3	DELETE.....	15
3.1.4	PUT (Update).....	15
3.2	Zdroj pro příklad je zde.....	16
3.3	Detekce cURL módu v requestu.....	17
3.4	http_response_code.....	17
3.5	Kódování asociativního pole.....	18
3.6	Funkce json_decode () PHP.....	18
4	htmlspecialchars.....	19
4.1	Připravené příkazy v MySQLi.....	19

4.1.1	Příklad (MySQLi s připravenými příkazy).....	20
-------	--	----

Připojení k databázi mysql

1 Mysqli

příklady http://www.koding.cz/popis.php?db=function&id_f=24

Objektově

Procedurálně

2 PDO (PHP Data Objects)

- PDO je rozšíření PHP (od verze 5.1) umožňující jednotným způsobem komunikovat s různými databázemi.

PDO podporuje

- DBLIB: FreeTDS / Microsoft SQL Server / Sybase
- Firebird (<http://firebird.sourceforge.net/>): Firebird/Interbase 6
- IBM (IBM DB2)
- INFORMIX - IBM Informix Dynamic Server
- MYSQL (<http://www.mysql.com/>): MySQL 3.x/4.0
- OCI (<http://www.oracle.com/>): Oracle Call Interface
- ODBC: ODBC v3 (IBM DB2 and unixODBC)
- PGSQL (<http://www.postgresql.org/>): PostgreSQL
- SQLITE (<http://sqlite.org/>): SQLite 3.x

Dostupní na <http://www.php.net/manual/en/book.pdo.php>.

- PDO je PHP rozhraní pro pohodlnější práci s SQL databází.
- Dlouhá léta bylo zvykem v PHP při komunikaci používat prostou funkci `mysql_query`.
- Té se předal SQL dotaz a tato funkce ho zavolala.
- Ošetření dat – [ošetření před SQL injection](#) bylo zcela na **programátorovi**.

2.1 Jaké přináší výhody a jak ho používat.

PDO

- vytvoření jednotného ovladače, který se z pohledu programátora bude chovat pokaždé stejně, bez ohledu na typ připojené databáze.
- Jednoduchá třída, která pro každou společnou vlastnost všech databází nabízí stejnou metodu ovládání.

- Některé metody jsou funkční jen pro některé databáze. Ukázalo se to jako velmi výhodné řešení nazvané
- je to objektové rozhraní databází pro PHP.
- Procedurální rozhraní budeme hledat marně.

2.1 Narušení SQL dotazu (SQL injection)

Další nebezpečí je, když nám útočník **modifikuje SQL dotaz**. Může se to stát, když výsledný text SQL dotazu skládáme třeba v PHP a v datech od útočníka ponecháme uvozovku, která ukončí *naši* původní podmínku a umožní se tak útočníkovi psát další vlastní příkazy.

Pokud do **nebezpečného kódu** zajišťujícího přihlášení uživatele:

zadej jméno a heslo:


```
$dotaz = "SELECT * FROM uzivatele WHERE
```

```
jmeno='{$_POST['jmeno']}' AND heslo='{$_POST['heslo']}'";
```

Pošleme jako `$_POST['heslo']` řetězec `' OR ''=''`, vznikne následující dotaz,

který útočníka přihlásí vždy, protože `'=''` bude vždy platné.

```
SELECT * FROM uzivatele WHERE jmeno='Jméno' AND heslo=' OR
```

```
'='' ';
```

2.1.1 Řešení

Při skládání SQL dotazů stačí ošetřit všechna data funkcí `mysql_real_escape_string`.

Escapuje speciální znaky v SQL dotazu, s ohledem na kódování spojení.

Příklad

```
<?php
```

```
$text = "Michel's computer";
```

```
$escape_text = mysql_real_escape_string($text);
```

```
printf("Escaped text: %s\n", $escape_text);
```

```
?>
```

Výsledek

```
Escaped text: Michel\'s computer
```

Zdroj: http://www.koding.cz/popis.php?db=function&id_f=24

Další možnost je využít tzv. [prepared statements](#),

```
<?php
```

```
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (?, ?)");
```

```
$stmt->bindParam(1, $name);
```

```
$stmt->bindParam(2, $value);
```

```
// insert one row
```

```

$name = 'one';

$value = 1;

$stmt->execute();

// insert another row with different values

$name = 'two';

$value = 2;

$stmt->execute();

?>

```

Kdy se místo proměnných do SQL dotazů píší jen otazníky a **bezpečné**

nahrazení zajistí až **databázová vrstva** (např. [Dibi](#), [PDO](#)).

2.2 PDO Připojení k MySQL

// Připojovací údaje

```

define('SQL_HOST', 'localhost');
define('SQL_DBNAME', 'database');
define('SQL_USERNAME', 'root');
define('SQL_PASSWORD', '');
$dsn = 'mysql:dbname=' . SQL_DBNAME . ';host=' . SQL_HOST . ";
$user = SQL_USERNAME;
$password = SQL_PASSWORD;
try {
    $pdo = new PDO($dsn, $user, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die('Connection failed: ' . $e->getMessage());
}

```

Zpracovávat chyby v SQL dotazech je možné pomocí `try – catch`. Nastavení `PDO::ATTR_ERRMODE`, které proběhlo přidáním atributu (`$pdo->setAttribute`) zajistí, že se v případě problému vyhodí výjimka.

2.3 Obyčejný dotaz

Nejjednodušší ekvivalent `mysql_query` je použití metody `query`. Ale není potřeba ho znát, dá se celkem obstojně vystačit s `prepare` a `execute` (popsány níže).

```
$dotaz = $pdo->query("SELECT jmeno, prijmeni FROM uzivatele");
```

V proměnné `$dotaz` potom bude výsledek dotazu.

2.4 Předpřipravený dotaz (prepared statement)

Asi hlavní výhoda oproti `mysql_*` funkcím jsou **předpřipravené dotazy**. Což automaticky při správném použití chrání před **SQL injection**.

2.4.1 SELECT

```
// Připravení dotazu
$dotaz = $pdo->prepare("SELECT jmeno, prijmeni FROM uzivatele WHERE id = ?");
// Vykonání dotazu
$vysledek = $dotaz->execute(array($idUzivatele));
```

Celé to pracuje tak, že na místa, kde chceme dosadit nějaké **proměnné z PHP** napíšeme jen otazníky. Vlastní data se potom předají jako pole metodě `execute`, která dotaz daty naplní a provede.

Nahrazování *otazníků* daty probíhá v pořadí dat v poli, které se předává do metody `execute`.

Kromě otazníků je možné používat i „PDO proměnné“ s dvojtečkou na začátku.

U komplikovanějších dotazů to **může zvýšit čitelnost** a přehlednost kódu.

```
// Připravení dotazu
```

```
$dotaz = $pdo->prepare("SELECT jmeno, prijmeni FROM uzivatele WHERE id =  
:idUzivatele");  
// Vykonání dotazu  
$vysledek = $dotaz->execute(array( ":idUzivatele" => $idUzivatele  
));
```

2.4.2 INSERT

```
// Přípravení dotazu  
$dotaz = $pdo->prepare("INSERT into uzivatele (jmeno, prijmeni) VALUES(?, ?);  
// Vykonání dotazu  
$vysledek = $dotaz->execute(array($jmeno, $prijmeni));
```

2.4.3 UPDATE

```
// Přípravení dotazu
$dotaz = $pdo->prepare("UPDATE uzivatele SET jmeno = ?, prijmeni = ? WHERE id = ?");
// Vykonání dotazu
$vysledek = $dotaz->execute(array( $jmeno, $prijmeni, $idUzivatele));
```

2.4.4 DELETE

```
// Přípravení dotazu
$dotaz = $pdo->prepare("DELETE FROM uzivatele WHERE id = ?");
// Vykonání dotazu
$vysledek = $dotaz->execute(array( $idUzivatele));
```

2.5 Procházení dat

Když potřebná data získáme z databáze, je potřeba je nějak zpracovat a vypsat v PHP.

2.5.1 Více řádků

```
$dotaz = $pdo->prepare("SELECT jmeno, prijmeni FROM uzivatele");
$dotaz->execute();
$uzivatele = $dotaz->fetchAll();

foreach ($uzivatele as $uzivatel) {
    echo $uzivatel["jmeno"] . " " . $uzivatel["prijmeni"];
}
```

2.5.2 Jeden řádek

```
$dotaz = $pdo->prepare("SELECT jmeno, prijmeni FROM uzivatele WHERE id = ?");
$dotaz->execute(array($idUzivatele));
$uzivatel = $dotaz->fetch();

echo $uzivatel["jmeno"] . " " . $uzivatel["prijmeni"];
```

2.5.3 Jedena položka

```
$dotaz = $pdo->prepare("SELECT 1 FROM uzivatele WHERE jmeno = ?");  
$dotaz->execute(array($jmenoUzivatele));  
$existujeJmeno = $dotaz->fetchColumn();  
  
echo $existujeJmeno ? "Jméno existuje" : "Jméno neexistuje";
```

Více informací o PDO: <http://php.net/manual/en/pdostatement.fetchcolumn.php>

2.6 Vícenásobný INSERT

Máme-li nějaké pole s hodně daty, které se mají najednou vložit do databáze. Je možné si buď nejprve připravit SQL dotaz (**\$pdo->prepare**) a v cyklu provádět **execute** s příslušnými daty, nebo si připravit dotaz plný otazníků a **provést ho najednou**.

```
$pripravaDotazu = "INSERT INTO uzivatele (jmeno, prijmeni) VALUES ";
$otazniky = array_fill(0, count($poleDat), "(?, ?)");
$pripravaDotazu .= implode(",", $otazniky);
//jan,Novak

$polozky = array();
foreach ($poleDat as $polozka) {
    array_push($polozky, $polozka['jmeno']);
    array_push($polozky, $polozka['prijmeni']);
}
$dotaz = $pdo->prepare($pripravaDotazu);
$vysledek = $dotaz->execute($polozky);
```

Co tento kód dělá?

- Nejprve připraví dotaz.
- Pro každou položku z `$poleDat` se připraví dva otazníky v dotazu.
- Potom se celé vícerozměrné pole projde a vytvoří se z něj jednoúroňové, které se předá do metody `execute`.

2.7 Příklad PDO

zobrazení dat

```
<?php
echo "<table style='border: solid 1px black;'>";
echo "<tr><th>Id</th><th>Firstname</th><th>Lastname</th></tr>";

class TableRows extends RecursiveIteratorIterator {
    function __construct($it) {
        parent::__construct($it, self::LEAVES_ONLY);
    }

    function current() {
        return "<td style='width:150px;border:1px solid black;'>" .
parent::current(). "</td>";
    }

    function beginChildren() {
        echo "<tr>";
    }

    function endChildren() {
        echo "</tr>" . "\n";
    }
}

$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDBPDO";

try {
    $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username,
$password);
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $stmt = $conn->prepare("SELECT id, firstname, lastname FROM MyGuests");
    $stmt->execute();

    // set the resulting array to associative
    $result = $stmt->setFetchMode(PDO::FETCH_ASSOC);
    foreach(new TableRows(new RecursiveArrayIterator($stmt->fetchAll())) as $k=>$
v) {
        echo $v;
    }
}
```

```

} catch(PDOException $e) {
    echo "Error: " . $e->getMessage();
}
$conn = null;
echo "</table>";
?>

```

2.8 Přidání dat pomocí PDO

https://www.w3schools.com/php/php_mysql_insert.asp

```

<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDBPDO";

try {
    $conn = new PDO("mysql:host=$servername;dbname=$dbname",
$username, $password);
    // set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";
    // use exec() because no results are returned
    $conn->exec($sql);
    echo "New record created successfully";
}
catch(PDOException $e)
{
    echo $sql . "<br>" . $e->getMessage();
}

$conn = null;
?>

```

2.9 Mazání dat pomocí PDO

https://www.w3schools.com/php/php_mysql_delete.asp

```

<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDBPDO";

try {
    $conn = new PDO("mysql:host=$servername;dbname=$dbname",
$username, $password);
    // set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // sql to delete a record
    $sql = "DELETE FROM MyGuests WHERE id=3";

    // use exec() because no results are returned
    $conn->exec($sql);
    echo "Record deleted successfully";
}
catch(PDOException $e)
{
    echo $sql . "<br>" . $e->getMessage();
}

$conn = null;
?>

```

2.10 Oprava dat pomocí PDO

```

<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDBPDO";

try {
    $conn = new PDO("mysql:host=$servername;dbname=$dbname",
$username, $password);
    // set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "UPDATE MyGuests SET lastname='Doe' WHERE id=2";

    // Prepare statement

```

```

$stmt = $conn->prepare($sql);

// execute the query
$stmt->execute();

// echo a message to say the UPDATE succeeded
echo $stmt->rowCount() . " records UPDATED successfully";
}
catch(PDOException $e)
{
    echo $sql . "<br>" . $e->getMessage();
}

$conn = null;
?>

```

3 REST API

REST (Representational State Transfer) –

architektura rozhraní, navržená pro distribuované prostředí.

Rozhraní REST je použitelné pro jednotný a snadný přístup ke zdrojům (resources). Zdrojem mohou být data, stejně jako stavy aplikace (pokud je lze popsat konkrétními daty). Všechny zdroje mají vlastní identifikátor [URI](#) a REST definuje čtyři základní metody pro přístup k nim.

3.1 Metody pro přístup ke zdrojům

REST implementuje čtyři základní metody, které jsou známy pod označením [CRUD](#), tedy vytvoření dat (Create), získání požadovaných dat (Retrieve), změnu (Update) a smazání (Delete). Tyto metody jsou implementovány pomocí odpovídajících metod HTTP protokolu.

3.1.1 GET (Retrieve)

Základní metodou pro přístup ke zdrojům je získání zdroje – metoda GET. Setkává se s ní každý uživatel webu dnes a denně – není to nic jiného než starý dobrý požadavek na stránku.

```
GET /api/user/pepa
```

```
Host: www.server.cz
```

3.1.2 POST (Create)

Získání dat je jednoduché a přímočaré. Stejně tak i vytvoření dat bude většině webových vývojářů připadat povědomé. Pro vytvoření dat slouží totiž metoda POST, známá (minimálně) z HTML formulářů.

Použití především pro přidávání dat

```
curl -u user:password -d "status=Zkousime REST API"
```

```
http://twitter.com/statuses/update.xml
```

3.1.3 DELETE

Zdroj lze smazat pomocí volání URI [HTTP metodou](#) DELETE. Volání je obdobné volání metody GET:

DELETE /api/user/pepa

Host: www.server.cz

3.1.4 PUT (Update)

Například:

```
curl -u user:password -d "url=http://zdrojak.root.cz"
```

```
http://twitter.com/account/update_profile.xml
```

3.2 Zdroj pro příklad je zde

<https://codeofaninja.com/2017/02/create-simple-rest-api-in-php.html>

<http://localhost/predn6/product/read.php>

GET <http://localhost/predn6/product/read.php> Params Send Save

Authorization Headers (1) Body Pre-request Script Tests Code

Type No Auth

Body Cookies Headers (8) Test Results Status: 200 OK Time: 50 ms

Pretty Raw Preview

```
{ "records": [{"id": "1", "name": "LG P880 4X HD", "description": "My first awesome phone!", "price": 336, "category_id": "3", "category_name": null}] }
```

<http://localhost/predn6/user/readall.php>

GET <http://localhost/predn6/user/readall.php> Params Send Save

Authorization Headers (1) Body Pre-request Script Tests Code

Type No Auth

Body Cookies Headers (8) Test Results Status: 200 OK Time: 50 ms

Pretty Raw Preview

```
{ "records": [{"id": "24", "login": "aaazxxxxxxxxxxxxxxxx", "heslo": "smaz", "jmeno": "aaazxxxxxxxxxxxxxxxx", "prijmeni": "smaz", "token": "", "role": "1", "id_organizace": "39"}, {"id": "20", "login": "smaz", "heslo": "smaz", "jmeno": "smaz", "prijmeni": "smaz", "token": "smaz", "role": "1", "id_organizace": "5"}, {"id": "19", "login": "test1koord", "heslo": "1234", "jmeno": "test1koord", "prijmeni": "1", "token": "", "role": "1", "id_organizace": "18"}, {"id": "18", "login": "test1reader", "heslo": "1234", "jmeno": "1", "prijmeni": "1", "token": "1", "role": "3", "id_organizace": "1"}, {"id": "16", "login": "test1", "heslo": "1234", "jmeno": "jan", "prijmeni": "Novak", "token": "1234", "role": "1", "id_organizace": "1"}, {"id": "13", "login": "bbaa", "heslo": "cvcb", "jmeno": "bbaa", "prijmeni": "cvcb", "token": "", "role": "1", "id_organizace": "10"}, {"id": "11", "login": "aaadad", "heslo": "heslo", "jmeno": "jan", "prijmeni": "Novu00e1k", "token": "1111", "role": "1", "id_organizace": "5"}, {"id": "9", "login": "test3", "heslo": "1234", "jmeno": "boba", "prijmeni": "dada", "token": "1111", "role": "1", "id_organizace": "3"}, {"id": "5", "login": "test2aaaa", "heslo": "1234", "jmeno": "test2aaaa", "prijmeni": "Novu00e1k", "token": "", "role": "1", "id_organizace": "18"}, {"id": "1", "login": "admin", "heslo": "1234", "jmeno": "admin", "prijmeni": "Novu00e1k", "token": "", "role": "0", "id_organizace": "1"}] }
```

Id_user	Login	Heslo	Jméno	Příjmení	role	id_organizace
24	aa	smaz	aa	smaz	1	1
16	test1	1234	jan	Novak	1	1
11	asdasd	heslo	jan	Novák	1	5
6	test3	1234	bbbb	dddddddddd	1	3
1	admin	1234	admin	Novák	0	2

PHP knihovna **cURL** představuje dobrý způsob, jak je možné stahovat data z cizího serveru.

Na základě dotazu sestaví HTTP request, který odešle na cílový server a po stažení obsahuje API pro (relativně) snadnou práci s daty.

Na rozdíl od nativní funkce **file_get_contents** (přes kterou můžeme také pokládat HTTP dotazy) nabízí mnohem lepší možnost konfigurace a stránky/soubory stahuje jako reálný prohlížeč.

Funkce **file_get_contents** interně knihovnu **cURL** používá, jenom nemá tak podrobné možnosti konfigurace.

3.3 Detekce cURL módu v requestu

Často se hodí detekovat, jestli byl aktuální požadavek položen prostřednictvím **cUrl**, nebo klasicky v prohlížeči.

V PHP pro to neexistuje přímá implementace, ale můžeme si napsat jednoduchou funkci sami:

```
/**
 * @return bool
 * @author Jan Barášek
 */
function isCurl(): bool
{
    return \stripos($_SERVER['HTTP_USER_AGENT'], 'curl') !== false;
}
```

Pokud disponujete Linuxem a jeho Terminálem, popřípadě jste na Macu, vyzkoušejte tento příkaz:

```
curl https://php.baraja.cz/curl
```

Příkaz provede interní request na tento web a vrátí výsledek.

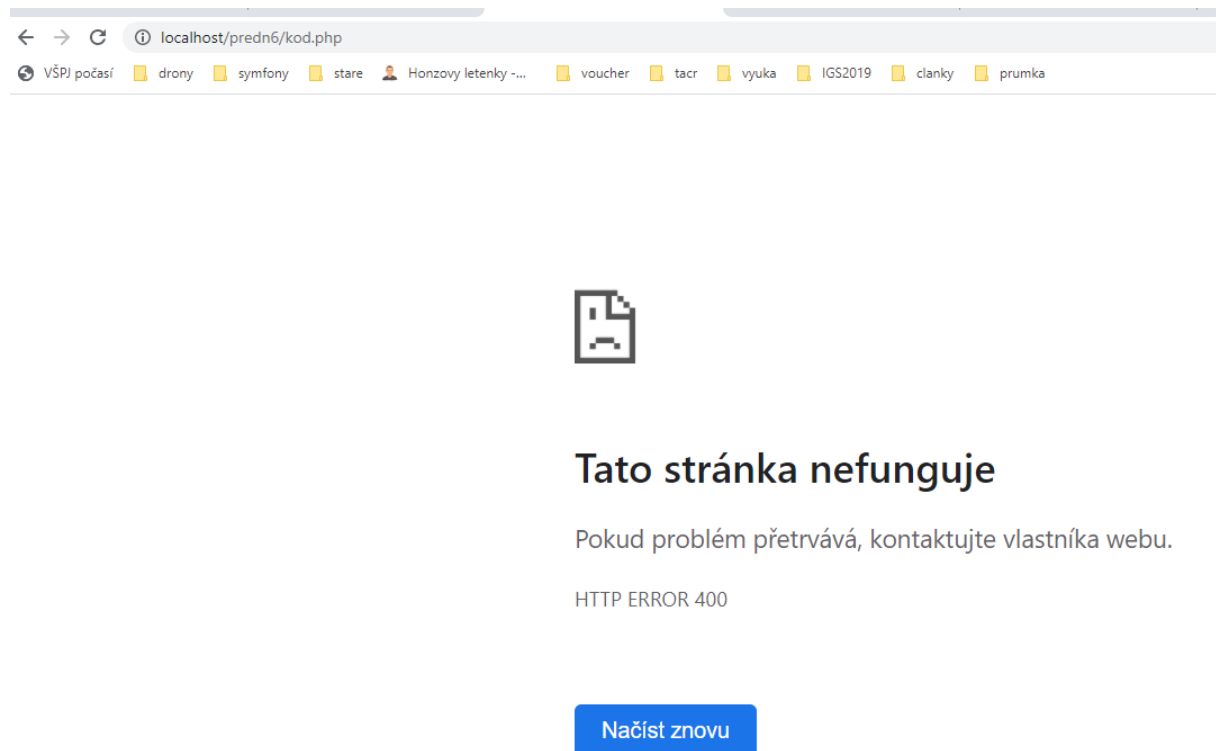
Kdyby aplikace neobsahovala detekce cURL requestu, bude vráceno HTML, jako kdyby šel request z prohlížeče. Protože se ale typy requestů detekují, nic nebrání tomu, abychom vrátili očištěný článek v Markdownu.

Výhodou jsou pak mnohem lépe očištěná data. Uživatel v prohlížeči zobrazíme naformátované HTML, zatímco robotovy jen základní obsah.

3.4 http_response_code

Nastavte stavový kód odpovědi HTTP na 404:

```
<?php
http_response_code(404);
?>
```



3.5 Kódování asociativního pole

```
<?php
$age = array("Peter"=>35, "Ben"=>37, "Joe"=>43);
```

```
echo json_encode($age);
?>
```

https://www.w3schools.com/php/showphp.php?filename=demo_json_encode1

Jak kódovat indexované pole do pole JSON:

```
<?php
$cars = array("Volvo", "BMW", "Toyota");

echo json_encode($cars);
?>
```

3.6 Funkce json_decode () PHP

```
<?php
$jsonobj = '{"Peter":35,"Ben":37,"Joe":43}';

var_dump(json_decode($jsonobj));
?>
```

https://www.w3schools.com/php/showphp.php?filename=demo_json_decode1

Ukládejte data JSON do proměnné PHP a poté je dekodujte do asociativního pole PHP:

```
<?php
$jsonobj = '{"Peter":35,"Ben":37,"Joe":43}';

var_dump(json_decode($jsonobj, true));
?>
```

https://www.w3schools.com/php/showphp.php?filename=demo_json_decode2

Jak přistupovat k hodnotám z objektu PHP:

```
<?php
$jsonobj = '{"Peter":35,"Ben":37,"Joe":43}';

$obj = json_decode($jsonobj);

echo $obj->Peter;
echo $obj->Ben;
echo $obj->Joe;
?>
```

https://www.w3schools.com/php/showphp.php?filename=demo_json_decode3

Jak přistupovat k hodnotám z asociativního pole PHP:

```
<?php
$jsonobj = '{"Peter":35,"Ben":37,"Joe":43}';
```

```
$arr = json_decode($jsonobj, true);
```

```
echo $arr["Peter"];
```

```
echo $arr["Ben"];
```

```
echo $arr["Joe"];
```

https://www.w3schools.com/php/showphp.php?filename=demo_json_decode4

4 htmlspecialchars

převéde speciální znaky v textu na HTML entity. Mezi speciální znaky patří: `&`, `'`, `"`, `<`, `>`

```
<?php
```

```
$s = 'Vítej ve fóru, tvůj problém vyřeší tag </body>, nemáš totiž ukončené body.';
```

```
echo htmlspecialchars($s);
```

4.1 Připravené příkazy v MySQLi

Následující příklad používá připravené příkazy a vázané parametry v MySQLi:

4.1.1 Příklad (MySQLi s připravenými příkazy)

```
<?php
```

```
$servername = "localhost";
```

```
$username = "username";
```

```
$password = "password";
```

```
$dbname = "myDB";
```

```
// Create connection
```

```
$conn = new mysqli($servername, $username, $password, $dbname);
```

```
// Check connection
```

```
if ($conn->connect_error) {
```

```
    die("Connection failed: " . $conn->connect_error);
```

```
}
```

```
// prepare and bind
```

```
$stmt = $conn->prepare("INSERT INTO MyGuests (firstname, lastname, email)  
VALUES (?, ?, ?)");
```

```
$stmt->bind_param("sss", $firstname, $lastname, $email);
```

```
// set parameters and execute
$firstname = "John";
$lastname = "Doe";
$email = "john@example.com";
$stmt->execute();

$firstname = "Mary";
$lastname = "Moe";
$email = "mary@example.com";
$stmt->execute();

$firstname = "Julie";
$lastname = "Dooley";
$email = "julie@example.com";
$stmt->execute();

echo "New records created successfully";

$stmt->close();
$conn->close();
?>
```

Řádky kódu k vysvětlení z výše uvedeného příkladu:

"INSERT INTO MyGuests (firstname, lastname, email) VALUES (?, ?, ?)"

V našem SQL vložíme otazník (?), Kde chceme nahradit celou hodnotu, řetězec, dvojitou nebo blobovou hodnotu.

Pak se podívejte na funkci bind_param ():

```
$stmt->bind_param("sss", $firstname, $lastname, $email);
```

Tato funkce váže parametry na dotaz SQL a říká databázi, jaké jsou parametry. Argument "sss" uvádí seznam typů dat, kterými jsou parametry. Znak s říká mysql, že parametr je řetězec.

Argument může být jednoho ze čtyř typů:

i - celé číslo

d - dvojnásobek

s - řetězec

b - BLOB

Pro každý parametr musíme mít jednu z nich.

Tím, že řekneme mysql, jaký typ dat očekávat, minimalizujeme riziko injekcí SQL.